

Majoration du nombre de mouvement nécessaire à la résolution du jeu `loopover.xyz`

Parent Quentin

Introduction

Le puzzle permutatif `loopover` (`loopover.xyz`) se présente comme une grille bidimensionnelle de dimensions $N \times N$ dont les lignes et les colonnes peuvent subir des permutations circulaires. À l'instar du Rubik's Cube, déterminer le Nombre de Dieu — le nombre maximal de mouvements nécessaires pour résoudre un état du puzzle — constitue un défi algorithmique et mathématique majeur. De par l'explosion combinatoire du nombre d'état (en $\Theta(N^2!)$). Dans cet article, nous proposons une approche théorique et algorithmique visant à établir une majoration du Nombre de Dieu, en particulier pour le cas $N = 5$. Notre méthodologie s'appuie sur la décomposition du groupe de permutations associé au puzzle ($\mathfrak{A}_{N^2}$ ou $\mathfrak{S}_{N^2}$). Nous modélisons la résolution via une stratégie par phases successives, consistant à isoler et résoudre des sous-blocs rectangulaires de taille $p \times q$. Nous démontrons que toute configuration valide d'une grille $N \times N$ peut être résolue en un nombre de mouvement polynomial en N , puis nous intéressons au cas particulier $N = 5$.

1 Jeu

1.1 Définition

[OBJECTIF] [DÉFINITION] [MVT/NOTATIONS]

1.2 Groupe de représentation du jeu

sg de $\mathfrak{S}_{N^2}$, engendré par blabla

À l'aide de commutateurs, on engendre les 3c [SCHÉMA] $\rightarrow$ contient $\mathfrak{A}_{N^2}$

- si N est pair, puisque U1 est de signature -1 , le groupe du puzzle est donc $\mathfrak{S}_{N^2}$
- si N est impair, toutes les mouvements élémentaires sont de signature 1, le groupe du puzzle est donc $\mathfrak{A}_{N^2}$

2 Résolution théorique

2.1 Minoration

Si $k = k_N$ est le maximum recherché, alors on peut associer à tout état du puzzle un mélange en moins de k mouvements.

Or, il existe $\sum_{i=0}^k (4N)^i$ mélanges en moins de k mouvements.

De plus, en retirant les mouvements inutiles (U1 D1 = id, par exemple), il n'y a que $(4N-1)$ choix à partir du second mouvement.

Finalement, il existe $1 + 4N \sum_{i=0}^{k-1} (4N-1)^i = 1 + 4N \frac{1 - (4N-1)^N}{2 - 4N}$.

On a donc $\frac{N^2!}{2} \leq 1 + 4N \frac{(4N-1)^{k-1}}{4N-2}$ (dans le cas N impair), d'où:

$$k_N \geq \frac{\log \left(\frac{N^2!-2}{4N} (N-1) + 1 \right)}{\log(4N-1)}$$

Pour $N = 5$, on obtient $k_5 \geq$

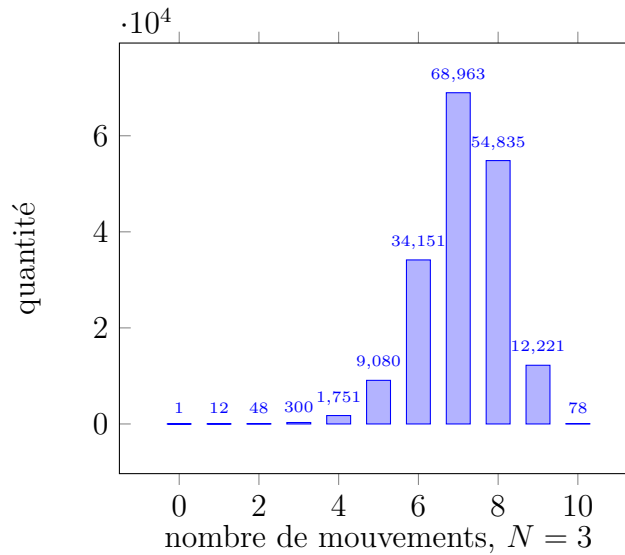
2.2 Algorithme de résolution

2.3 Majoration

3 Résolution algorithmique

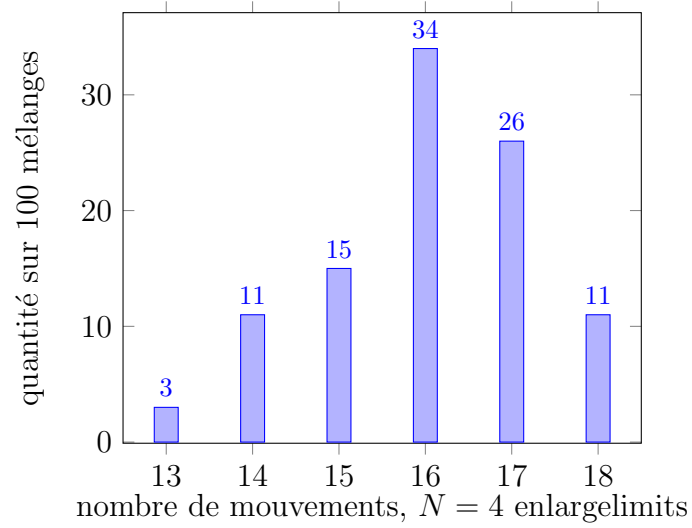
3.1 IDA*

Résultats



Pour $N = 3$, en explorant à l'aide d'un algorithme de Heap tous les mélanges (de signature 1) que $k_3 = 10$.

Cependant, sur un échantillon de 100 mélanges aléatoires sur un puzzle de dimension 4, on observe un temps moyen de 14.61 secondes pour la résolution. En explorant de même tous les états du puzzle, l'ordinateur prendrait 9 milliards d'année pour atteindre un résultat, rendant cette solution impraticable.



Pour $N = 5$, l'algorithme IDA* ne termine pas en une demie-heure, nous empêchant également d'utiliser cette solution.

3.2 Résolution multiphase

Motivation

On vérifie de même qu'en 1.2 que le sous-groupe $\{\sigma \in G \mid \text{le bloc } p * q \text{ est résolu}\}$ est égal à $\langle R_i, U_j \rangle_{i > p, j > q}$, permettant donc de chercher une solution n'utilisant que ces mouvements, réduisant donc les états explorés lors de l'utilisation de l'algorithme IDA*.

Résultats

4 Conclusion